

第6章 面向对象

类与对象 · 继承与 MRO · 鸭子类型 · 运算符重载

本章目标:用 C 的 struct 与函数指针为锚点,讲透 Python 的类、self、继承与方法解析顺序(MRO)、鸭子类型、运算符重载,以及生产中最实用的 classmethod/staticmethod/property 三件套。学完后你能独立设计出结构清晰、可测试、可扩展的业务类。

直觉起点:在 C 里,数据和操作它的函数是分离的:struct 存数据,函数接受 struct 指针。Python 把两者合二为一——对象自带方法,方法第一个参数就是"那个对象自己"。语法变了,但底层思想依然是"数据 + 行为"。

前置要求:第1章名字绑定(对象 vs 名字)、第4章函数(默认参数、闭包)、第5章引用计数(对象生命周期)。

本章路线:第一阶段(2 小时)读 §6.1–§6.4,动手写第一个类并理解继承;第二阶段(1.5 小时)读 §6.5–§6.8,掌握鸭子类型与运算符重载;第三阶段(2 小时)完成章末实验。

本章知识导图

- └─ 6.1 从 struct 到 class —— 数据与行为的合一
- └─ 6.2 self 与 __init__ —— 第一个类、名称改写
- └─ 6.3 属性控制 —— _ 约定 / @property / __slots__
- └─ 6.4 继承与 MRO —— super、C3 线性化
- └─ 6.5 鸭子类型 —— 多态的真面目
- └─ 6.6 运算符重载 —— __eq__ / __lt__ / __repr__ 及配套
- └─ 6.7 三种方法 —— 实例方法 / @classmethod / @staticmethod
- └─ 6.8 组合优于继承 —— 生产设计取向

§6.1 从 struct 到 class

C 程序员描述"一个学生"时,写一个 struct,再写一组接受该指针的函数;调用时手传 `&stu`。Python 的 class 把这两个动作合成一个声明:字段与方法长在同一个命名空间里,方法调用时解释器自动把对象本身作为第一个参数传进去。

C(struct + 函数)	Python(class)
<code>struct Student s; init(&s, ...);</code>	<code>s = Student(...)</code>
操作函数在 struct 外,靠命名前缀归类	方法在类体内,归属关系由语法保证
忘记初始化→未定义行为	<code>__init__</code> 保证创建即初始化
函数指针成员模拟"方法",绕且晦涩	方法就是普通函数,第一个参数是 self
struct 拷贝是值拷贝	赋值只是复制引用,对象只有一个

类(class)是蓝图,**实例(instance)**是对象。类本身也是对象(第11章会深入),因此"定义类"只是创建一个可调用的、负责生产实例的工厂。

6.1.1 从"结构体+函数"到"类"

C 写法是数据与逻辑分离的;Python 写法把它们收进一个类,数据不出类也能被类方法加工。两者解决的问题相同:把一组相关的数据与操作绑定。

从 C 迁移的开发者常纠结“为什么不保持纯数据”。实际上 Python 的对象模型恰是 C 结构化数据的自然延伸:struct 表达“数据长什么样”,class 再附加“这些数据能做什么”。合并之后,数据与操作的匹配由解释器强制,而不是靠命名规范提醒,漏调 init 之类的问题在语法层面就被堵死。

§6.2 self 与 __init__:你的第一个类

```
class Point:
    """二维点。__init__ 负责初始化,__repr__ 负责调试展示"""

    def __init__(self, x: float, y: float) -> None:
        self.x = x
        self.y = y

    def distance(self, other: "Point") -> float:
        """两点间欧氏距离。注意 self 是第一个参数"""
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx * dx + dy * dy) ** 0.5

    def __repr__(self) -> str:
        return f"Point({self.x}, {self.y})"

p1 = Point(1.0, 2.0)          # 等价于 Point.__init__(p1, 1.0, 2.0)
p2 = Point(4.0, 6.0)
print(p1.distance(p2))       # 5.0
print(p1)                    # Point(1.0, 2.0)
```

注意 `Point(1.0, 2.0)` 这种构造语法之所以成立,正是因为 `__init__` 接收了实例本身并完成初始化——理解这一行,就理解了 Python 一切魔法方法的入口。

关键理解: `p1.distance(p2)` 只是 `Point.distance(p1, p2)` 的语法糖——Python 没有“this 隐式传入”这回事,self 就是一个普普通通的参数,因此**定义时必须写 self,调用时绝不能传 self**。

误区:忘写 self 或调用时多传了参数。定义 `def greet(): ...` 而调用 `obj.greet()` 会报“takes 0 positional arguments but 1 was given”——因为解释器把 `obj` 硬塞给了第一个参数。另外记住 `__init__` 必须返回 `None`,显式 `return` 一个值会抛 `TypeError`。

6.2.1 __init__ 不是构造器

真正的构造器是 `__new__` (负责分配内存), `__init__` 只负责“初始化已有对象”。日常开发只写 `__init__` 即可;只有实现单例、不可变对象或元类时才会碰 `__new__`。`__init__` 可以省略:没写时 Python 调用 `object` 的默认初始化。

6.2.2 名称改写:双下划线不是“私有”

Python 没有 `private` 关键字,只有约定与改名。`_x` 表示“内部实现,勿外部访问”(纯约定,照常可访问);`__x` 会触发名称改写,变成 `_ClassName__x`,主要用来避免子类不小心覆盖父类属性:

```

class BankAccount:
    def __init__(self, owner: str) -> None:
        self.owner = owner
        self.__balance = 0          # 改写为 _BankAccount__balance

    def deposit(self, amount: float) -> None:
        self.__balance += amount

acct = BankAccount("李雷")
acct.deposit(100)
print(acct.__balance)              # AttributeError!
print(acct._BankAccount__balance)  # 100 — 改写可查,但别这么写

```

生产规范:内部成员用单下划线 `_xxx`;双下划线仅用于"防止子类冲突"的场景(如混入类)。真正的访问控制交给 `@property`。

§6.3 属性控制:@property 与 `__slots__`

C 的 `struct` 字段谁都能读写;生产 Python 用 **@property** 把"读取"包装成方法调用,从而可以在赋值时做校验、在读取时做计算,调用方语法不变:

```

class Employee:
    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self._salary = 0.0
        self.salary = salary        # 走 setter 校验

    @property
    def salary(self) -> float:
        """只读属性与 setter 同名,调用方像访问字段"""
        return self._salary

    @salary.setter
    def salary(self, value: float) -> None:
        if value < 0:
            raise ValueError("工资不能为负")
        self._salary = value

    @property
    def annual(self) -> float:
        return self._salary * 12    # 计算属性,无 setter → 只读

e = Employee("韩梅梅", 8000)
e.salary = 9000
print(e.annual)                    # 108000
e.salary = -1                      # ValueError: 工资不能为负

```

判断该用 `property` 还是裸字段:先写裸字段,等"需要校验/计算/缓存"时再升级为 `property`——调用方代码一行不用改。这比 C 里改字段为宏/函数要平滑得多。

6.3.1 __slots__:省内存的字段白名单

```
class User:
    """默认每个实例都有 __dict__ 字典;__slots__ 改为固定描述符"""
    __slots__ = ("name", "age")

    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age

u = User("王五", 30)
u.email = "w@example.com"          # AttributeError: 没有这个槽
```

百万级小对象场景用 __slots__ 能显著降内存(省掉每实例的 dict 与弱引用表);代价是失去动态加属性。先测量再优化,别默认使用。

§6.4 继承与方法解析顺序

继承让子类复用父类行为并覆盖差异。调用 `super().__init__(...)` 是子类初始化父类部分的唯一正道——忘记调用会让父类字段缺失,这是继承场景第一大坑。

```
class Shape:
    def __init__(self, name: str) -> None:
        self.name = name

    def area(self) -> float:
        raise NotImplementedError("子类必须实现 area")

    def describe(self) -> str:
        return f"{self.name} 面积 {self.area():.2f}"

class Rectangle(Shape):
    def __init__(self, w: float, h: float) -> None:
        super().__init__("矩形")      # 必须先初始化父类部分
        self.w = w
        self.h = h

    def area(self) -> float:
        return self.w * self.h

r = Rectangle(3.0, 4.0)
print(r.describe())                  # 矩形 面积 12.00
```

误区:多继承里忘了调用链。 `super()` 不是"调父类",而是"沿 MRO 找下一个合作者"。菱形继承(A 有 B、C 两个子类,D 同时继承 B、C)中,如果 B、C 都调 `super().__init__`,D 实例化时 A.__init__ 恰好执行一次;如果谁漏了,初始化链就断。
规则:多继承里每一层都要调用 `super().__init__()`。

6.4.1 MRO:C3 线性化

Python 用 C3 算法把继承关系拍平成一条唯一的方法查找顺序,存在 `ClassName.__mro__` 里。查找方法时按 MRO 从左到右找第一个命中的。这与 C++ 的虚函数表思路不同:

C++(虚函数表)	Python(MRO)
编译期生成虚表,运行期查表	运行期沿 <code>__mro__</code> 线性顺序查找
多继承二义性要 virtual 显式处理	C3 保证唯一顺序,super() 自动接力
覆盖发生在编译期绑定	方法在调用那一刻动态决定(鸭子类型)
虚表指针藏在对象里,内存开销固定	查找是运行期遍历,慢在"每次调用"

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

# C3 保证:每个类只出现一次,且顺序全局一致
for cls in D.__mro__:
    print(cls.__name__, end=" ")
print()                                     # 输出: D B C A object
```

生产建议:单继承 + 混入(mixin)足以覆盖 95% 需求;三层以上的深继承或"歪菱形"会让 MRO 与直觉脱节,优先改为组合。

§6.5 鸭子类型:多态的真面目

C 的多态靠继承 + 虚函数;"长得像就用"是 Python 的多态。检查"是否支持某操作"时,不要先 `isinstance` 再分支,而是直接调用并让错误自然发生——这叫 EAFP(easier to ask forgiveness than permission):

```
def total_area(shapes: list) -> float:
    """不检查类型,只要求每个元素有 area() 方法"""
    return sum(s.area() for s in shapes)

class Circle:
    # 完全不继承 Shape!
    def __init__(self, r: float) -> None:
        self.r = r

    def area(self) -> float:
        return 3.14159 * self.r * self.r

items = [Rectangle(3.0, 4.0), Circle(2.0)]
print(total_area(items))                # 12.0 + 12.57 = 24.57
```

C(显式类型分支)	Python(EAFP)
<code>if (shape->type == CIRCLE) ...</code>	直接调用 <code>s.area()</code> ,没有就抛 <code>AttributeError</code>
新增类型要改 <code>switch</code> 与头文件	新增类型只要方法名对上,调用方零改动
编译器在编译期抓住类型错误	错误推迟到运行期——用测试与静态检查工具兜底

误区:到处 `isinstance` 检查。 `isinstance` 分支写多了,代码会退化成 C 式 `switch`,每加一个类型改一个函数。正确姿势:接口约定 + 文档写明"需要 `area()` 方法",必要时用第10章的 `Protocol` 做静态检查,运行期直接调用。

§6.6 运算符重载:让对象参与表达式

C 的运算符只对内置类型生效;Python 允许类定义运算符行为。生产中最常用的是排序与比较: `__lt__` 让对象能被 `sorted/max/min` 使用, `__eq__` 让对象可比较可哈希:

```
from functools import total_ordering

@total_ordering          # 只实现 __eq__ 和 __lt__,其余自动补全
class Score:
    def __init__(self, name: str, value: int) -> None:
        self.name = name
        self.value = value

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Score):
            return NotImplemented
        return self.value == other.value

    def __lt__(self, other: "Score") -> bool:
        return self.value < other.value

    def __repr__(self) -> str:
        return f"{self.name}:{self.value}"

scores = [Score("甲", 88), Score("乙", 95), Score("丙", 79)]
print(sorted(scores))          # [丙:79, 甲:88, 乙:95]
print(max(scores))             # 乙:95
```

误区:实现 `__eq__` 却忘了 `__hash__`。 Python 规定:定义 `__eq__` 的类自动失去默认哈希(变为不可哈希),放进 `set/dict` 的 `key` 会直接 `TypeError`。要么同时实现 `__hash__` (基于同一组字段),要么让对象明确不可哈希(当作"不该进集合"的信号)。

另一个高频重载是 `__repr__` (给开发者看的调试信息)与 `__str__` (给用户看的展示)。只写一个就写 `__repr__`,因为 `str()` 会退回 `repr`,而 f-string 优先用 `__str__`。

§6.7 三种方法:实例方法 / `@classmethod` / `@staticmethod`

类体里可以出现三类函数,用途截然不同:

```

class Config:
    env = "prod" # 类属性,所有实例共享

    def __init__(self, key: str) -> None:
        self.key = key # 实例方法:操作单个实例

    def display(self) -> str:
        return f"{self.key}={Config.env}" # 实例方法第一参是 self

    @classmethod
    def set_env(cls, env: str) -> None:
        """类方法:第一参是类本身,常用于工厂与全局配置"""
        cls.env = env

    @staticmethod
    def is_valid(key: str) -> bool:
        """静态方法:不接 self/cls,只是逻辑上归属本类"""
        return bool(key) and key.isascii()

Config.set_env("staging") # 通过类调用
dev = Config("api_token")
print(dev.display()) # api_token=staging
print(Config.is_valid("ok")) # True

```

选用口诀:需要操作实例数据→实例方法;需要访问或修改类属性/写工厂→@classmethod;与实例、类都无关、只是收纳→@staticmethod。@staticmethod 在新代码里频率很低,多数"工具方法"放模块级函数更自然。

6.7.1 classmethod 工厂

经典生产用法:一个类有多个合法构造路径,用 classmethod 提供带名字的工厂:

```

from datetime import date

class Person:
    def __init__(self, name: str, birth: date) -> None:
        self.name = name
        self.birth = birth

    @classmethod
    def from_iso(cls, name: str, iso: str) -> "Person":
        """从 ISO 字符串构造,失败时抛 ValueError,职责集中"""
        return cls(name, date.fromisoformat(iso))

    @property
    def age(self) -> int:
        today = date.today()
        return today.year - self.birth.year - (
            (today.month, today.day) < (self.birth.month, self.birth.day))

p = Person.from_iso("赵六", "2000-03-15")
print(p.age)

```

§6.8 组合优于继承:生产设计取向

继承表达"is-a",组合表达"has-a"。继承树深了会僵硬:改父类影响所有子孙、子类被迫继承不需要的的方法。生产代码里,优先用组合:把变化的行为塞进一个对象,通过构造注入:

```
class Logger:                                # 策略:可替换的行为
    def log(self, msg: str) -> None:
        print(f"[log] {msg}")

class SilentLogger:                          # 另一策略,同一接口
    def log(self, msg: str) -> None:
        pass

class Service:
    def __init__(self, logger: Logger) -> None:
        self.logger = logger                # 组合:行为注入,而非继承

    def run(self) -> None:
        self.logger.log("开始工作")

Service(Logger()).run()                      # 打印
Service(SilentLogger()).run()               # 静默—调用方代码不变
```

判断何时继承:①严格 is-a(经理是员工);②需要复用实现且差异有限;③多态入口(接口/协议)。出现"只想借用两个方法""父类一半方法用不上""为了调 super()"时,改组合。

工程建议 1:给类写 `__repr__` 是免费的调试投资——异常信息、日志、pytest 失败输出都会打印它。宁可少写文档字符串,也别省 `__repr__`。

工程建议 2:类内部不直接碰"环境"细节(打印、读环境变量、连数据库),通过构造注入依赖,这样单测时替换一个假的实现即可,这也是"组合优于继承"的直接收益。

本章速查表

场景	写法
定义类	<code>class Point: ...</code>
初始化	<code>def __init__(self, x): self.x = x</code>
方法第一参	定义写 <code>self</code> ,调用绝不传
内部成员	单下划线 <code>_x</code> (约定)
防子类覆盖	<code>__x</code> → 改写为 <code>_Class__x</code>
只读/校验字段	<code>@property</code> + <code>@x.setter</code>
省内存白名单	<code>__slots__ = ("a", "b")</code>
子类初始化父类	<code>super().__init__(...)</code>
查看查找顺序	<code>D.__mro__</code>
比较排序	<code>__eq__</code> + <code>__lt__</code> ,配 <code>@total_ordering</code>
哈希配套	实现 <code>__eq__</code> 必须同时实现 <code>__hash__</code>
调试展示	<code>__repr__</code> (str 会退回它)
工厂构造	<code>@classmethod</code> 返回 <code>cls(...)</code>
纯工具函数	<code>@staticmethod</code> 或模块级函数
多态检查	直接调用,EAFP,少用 <code>isinstance</code>
扩展方式	组合注入优先于加深继承

最小必会清单

1. 能写一个带 `__init__`、实例方法、`__repr__` 的类,并解释 `p1.distance(p2)` 的完整调用过程
2. 说出 `_x` 与 `__x` 的区别,并正确使用 `@property` 实现只读字段
3. 会用 `super().__init__` 初始化继承链,并画出单继承与一次混入的 MRO
4. 能让自定义类直接参与 `sorted()`:实现 `__lt__` 并配齐 `__eq__`/`__hash__`
5. 能区分实例方法、`@classmethod`、`@staticmethod` 三种场景,并写出一个 `classmethod` 工厂
6. 能说明"组合优于继承"的理由,并把一个"为复用继承"的类重构为注入

自测题

Q

Q1. `p1.distance(p2)` 与 `Point.distance(p1, p2)` 是什么关系?定义方法时漏写 `self` 会怎样?

答案:完全等价,前者是后者的语法糖。漏写 `self` 后,调用 `p1.distance(p2)` 会把 `p1` 当作第一个实参,抛 `"takes 1 positional argument but 2 were given"`;若只有 `p1.distance()` 这种调用,则 `p1` 被当作参数,方法体内访问属性时抛 `AttributeError`。

Q

Q2. 类里定义 `__x = 1` 后,外部访问 `obj.__x` 报 `AttributeError`,为什么?如何访问?

答案:双下划线开头触发名称改写, `__x` 在类体内被改成 `_类名__x`。外部应访问 `obj._类名__x`——但这是破坏封装的行为,生产代码不这么做;需要"私有"语义时用单下划线 + `@property`。

Q

Q3. 为什么定义 `__eq__` 之后对象会变得不可哈希?如何让对象既能比较又能进 `set`?

答案:Python 默认哈希基于 `id`;一旦定义 `__eq__`,默认哈希被停用以保持"相等对象哈希必相同"的不变量。同时实现 `__hash__` (用 `__eq__` 比较的同一组字段,如 `return hash((self.value,))`),对象即可安全进入 `set/dict`。

Q

Q4. 菱形继承 `D(B, C)` 中, `B`、`C` 都继承 `A` 且各自 `__init__` 调用了 `super().__init__()`, `D` 实例化时 `A.__init__` 执行几次? 如果 `B` 漏调 `super()` 会怎样?

答案:一次。MRO 把每个类在线性化列表里只放一次, `super()` 沿 MRO 接力, `A` 只被初始化一次。 `B` 漏调则初始化链在 `B` 处断裂, `super().__init__` 之后的合作者(`A` 与 `C` 的初始化)全部被跳过,对象处于部分初始化状态。

Q

Q5. 一段代码里频繁出现 `if isinstance(x, A): ... elif isinstance(x, B): ...`,它的问题是什么?怎么改?

答案:它把"类型分支"硬编码进调用方,新增类型要改调用方,违背开闭原则,与鸭子类型哲学相悖。改法:定义统一接口(同名方法或 Protocol),调用方直接调用;若分支本质是算法族,用策略对象注入(\$6.8 的组合模式)。

Q

Q6. `@property` 与裸字段相比多了函数调用开销,为什么生产代码仍然大量使用?

答案:因为收益大于开销:①可以在赋值点集中校验,错误提前到"写入时"而非"使用时";②可以做成计算属性,调用方无感;③可以只读或按条件隐藏底层字段,把"如何存"与"如何暴露"解耦;④升级成本低——先裸字段,需要时再包 `property`,调用方代码零改动。性能敏感的热路径才需要避开它。

章末实验题:员工薪酬系统

实验 6.1 员工薪酬系统(覆盖:\$6.2–\$6.8)

需求:① 基类 `Employee` 含姓名、基本工资, `pay()` 返回月薪, `describe()` 输出简介;② 子类 `Manager` 按级别给系数,子类 `Salesman` 按业绩提成;③ 工资为 `property`,负值报错;④ 员工支持 `==`(同名同类型)与 `<`(按月薪),可直接 `sorted`;⑤ 用 `classmethod` 工厂从字典构造;⑥ 组合一个 `Department` 管理多名员工,提供 `total_pay()` 与 `top_pay()`;⑦ 写 `main` 演示排序与多态统计。

覆盖知识点:类定义与 `__init__`(\$6.2)/名称改写与 `property`(\$6.2、\$6.3)/继承与 `super`(\$6.4)/鸭子类型与多态(\$6.5)/运算符重载与 `total_ordering`(\$6.6)/`classmethod` 工厂与三种方法(\$6.7)/组合(\$6.8)。

实验环境:Python 3.10+,单文件 `payroll.py`。

```

# payroll.py — 员工薪酬系统(覆盖第6章全部知识点)
from functools import total_ordering

@total_ordering
class Employee:
    """员工基类:月薪属性带校验,支持比较与排序"""

    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self.salary = salary          # 走 setter 校验

    @property
    def salary(self) -> float:
        return self._salary

    @salary.setter
    def salary(self, value: float) -> None:
        if value < 0:
            raise ValueError(f"工资不能为负: {value}")
        self._salary = value

    def pay(self) -> float:
        return self._salary

    def describe(self) -> str:
        return f"{self.__class__.__name__} {self.name} 月薪 {self.pay():.0f}"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Employee):
            return NotImplemented
        return (self.name, type(self)) == (other.name, type(other))

    def __lt__(self, other: "Employee") -> bool:
        return self.pay() < other.pay()

    def __repr__(self) -> str:
        return self.describe()

    @classmethod
    def from_dict(cls, data: dict) -> "Employee":
        """工厂:按 kind 分派到子类,职责集中在类方法里"""
        if data["kind"] == "manager":
            return Manager(data["name"], data["salary"], data["level"])
        if kind == "sales":
            return Salesman(data["name"], data["salary"], data["sales"],
                             data.get("rate", 0.05))
        raise ValueError(f"未知员工类型: {kind}")

class Manager(Employee):
    """经理:级别越高,月薪系数越大"""

    def __init__(self, name: str, salary: float, level: int) -> None:
        super().__init__(name, salary) # 先初始化父类部分

```

```
self.level = level
```

```
def pay(self) -> float:  
    return self.salary * (1 + 0.2 * self.level)    # 多态覆盖
```

```
class Salesman(Employee):  
    """销售:底薪 + 提成"""  
  
    def __init__(self, name: str, salary: float,  
                  sales: float, rate: float = 0.05) -> None:  
        super().__init__(name, salary)  
        self.sales = sales  
        self.rate = rate  
  
    def pay(self) -> float:  
        return self.salary + self.sales * self.rate  
  
class Department:  
    """组合:部门拥有多名员工,对外提供统计能力"""  
  
    def __init__(self, name: str, members: list | None = None) -> None:  
        self.name = name  
        self.members = list(members or [])  
  
    def add(self, emp: Employee) -> None:  
        self.members.append(emp)  
  
    def total_pay(self) -> float:  
        return sum(e.pay() for e in self.members)  
  
    def top_pay(self) -> Employee:  
        return max(self.members)    # 依赖 __lt__  
  
def main() -> None:  
    raw = [  
        {"kind": "manager", "name": "李雷", "salary": 20000, "level": 2},  
        {"kind": "sales", "name": "韩梅梅", "salary": 8000,  
         "sales": 60000, "rate": 0.06},  
        {"kind": "sales", "name": "王五", "salary": 8000, "sales": 30000},  
    ]  
    dept = Department("销售部")  
    for item in raw:  
        dept.add(Employee.from_dict(item))    # 工厂 + 多态  
  
    for emp in sorted(dept.members):    # 排序走 __lt__  
        print(emp.describe())  
    print(f"部门总薪酬: {dept.total_pay():.0f}")  
    print(f"薪酬最高: {dept.top_pay()}")  
  
if __name__ == "__main__":  
    main()
```

设计说明:① from_dict 工厂把 "反序列化分派" 收进类体系,新增工种只改工厂一个函数;② pay() 在子类覆盖、describe() 复用多态,体现"继承用于复用实现、多态用于扩展行为";③ __lt__ 让 max/sorted 直接可用,total_ordering 自动补齐 >、<=、>=;④ Department 用组合容纳任意 Employee,若将来换成接口或 Protocol,这里不用动;⑤ 校验集中在 property setter,非法数据在构造时就失败,而不是运行到 pay 时才爆。

下一章预告:第7章 文件与数据交换——pathlib 路径操作、open 的编码世界、JSON/CSV 结构化数据、按行流式处理大文件。学完面向对象,让数据在磁盘与内存之间安全流动。